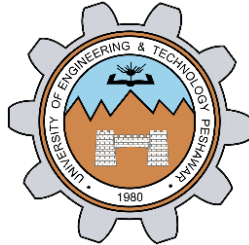


Shortest Path Problem Using Dijkstra's Algorithm
Open Ended Lab/Project Report



Spring 2026
CSE-210L Data Structures and Algorithms

Submitted by: **M Mohsin Khan**

Registration No.: **24PWCSE2382**

Class Section: **C**

Submitted to:

Engr. Abdullah Hamid

7 July, 2026

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Table of Contents

A) Problem Understanding	2
B) Design Decisions	3
1. Graph Representation → Adjacency List.....	3
2. Priority Queue → STL priority_queue	3
3. Input Method → Interactive Console Input	3
4. Weight Semantics → Distance/Cost	3
5. Output Format → Path and Total Cost.....	3
C) Extension Challenge → Dynamic Road Blocking (Challenge C).....	4
D) Algorithm Walkthrough	4
F) Test Cases	5
Test Case 1 — Normal Case: Finding the Shortest Route.....	5
Test Case 2 — Edge Case: Disconnected Graph.....	5
Test Case 3 — Complex Case: Dynamic Road Blocking.....	5
Test Case 4 — Invalid Node Input	6
Test Case 5 — Same Source and Destination.....	6

A) Problem Understanding

The shortest path problem is widely used in navigation systems, delivery services, and route planning applications. It involves finding the minimum-cost path between two locations in a weighted graph. Dijkstra's Algorithm is an efficient algorithm for solving this problem when all edge weights are non-negative.

In this project, I developed a **Smart Route Planner** where locations (e.g., Hostel, Madina Market, UET, or any set of user-defined places) are represented as nodes, and roads between them are represented as weighted edges indicating travel cost (distance/time, depending on how weights are entered).

The program takes the graph directly from the user through the console — number of locations, their names, number of roads, and each road's endpoints and weight. After selecting a source and destination, the program runs Dijkstra's Algorithm to compute the shortest route and displays the full path along with its total cost. It also supports **dynamic road blocking**: a road can be removed from the graph at runtime, and the shortest path is recalculated on the updated graph without rebuilding it from scratch. The program also handles invalid location names, negative weights, and unreachable destinations. This project demonstrates the practical application of Dijkstra's Algorithm to route planning, including how real-time changes (like a blocked road) can be handled efficiently.

B) Design Decisions

1. Graph Representation → Adjacency List

A graph can be represented using either an adjacency matrix or an adjacency list. I chose an **adjacency list** because road networks are sparse — each location connects to only a few others, not every other location. An adjacency list avoids wasting memory on non-existent roads and makes edge removal (for road blocking) a simple, localized operation.

In my implementation, the graph is stored as `vector<vector<pair<int,int>>> adjList`, where `adjList[u]` holds a list of `{neighbor_index, weight}` pairs for node `u`. Location names are stored separately in a `vector<string> names`, and a location's name is converted to its index using `find()` before being used in the graph. I used integer indices instead of strings directly in the adjacency list (unlike a `map<string, ...>` approach) because integer-indexed vectors are faster to access and simpler to iterate over when running Dijkstra's Algorithm.

2. Priority Queue → STL `priority_queue`

Dijkstra's Algorithm needs a structure that always gives back the unvisited node with the smallest known distance. I used the **STL `priority_queue`** instead of building a custom min-heap, because it is already efficient and well-tested, letting me focus on implementing Dijkstra's Algorithm correctly rather than heap mechanics. Since `priority_queue` is a max-heap by default, I used `priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>>` to turn it into a min-heap, ordering by distance first.

3. Input Method → Interactive Console Input

Graph data can come from manual input, a hardcoded graph, or a file. I chose **interactive console input** because it lets any user build and test a completely custom map at runtime without editing or recompiling the code. The program first asks for the number of locations and their names, then the number of roads and each road's from, to, and weight, validating each entry (rejecting unknown location names and negative weights) before adding it to the graph.

4. Weight Semantics → Distance/Cost

Edge weights can represent distance, time, or cost. I treated weights generically as **travel cost** (the user decides whether it represents kilometers, minutes, or fuel cost when entering data), since the algorithm's logic is identical regardless of what unit the number represents. This keeps the implementation flexible without hardcoding a specific unit.

5. Output Format → Path and Total Cost

I chose to display the **complete path** (sequence of location names) and the **total cost** as a single number, since this gives the user both the route and a clear quantitative result. After Dijkstra's Algorithm computes `dist[]` and `parent[]`, the `buildPath()` function reconstructs the route by walking backward from the destination through `parent[]`, then reversing it, and `main()` prints it directly to the console.

C) Extension Challenge → Dynamic Road Blocking (Challenge C)

Among the challenges offered (graph visualization, performance analysis, turn-by-turn directions, and dynamic edge updates), I implemented **Dynamic Road Blocking** because road closures are a realistic scenario in navigation systems, and my adjacency-list representation makes this efficient to support.

Instead of rebuilding the graph, the `blockRoad(adjList, u, v)` function removes the edge between two nodes directly: it searches `adjList[u]` for the entry pointing to `v` and erases it, then does the same in `adjList[v]` for `u` (since the graph is undirected, each edge exists in both directions and must be removed from both). After a road is blocked, choosing "Find shortest path" again simply calls `dijkstra()` on the same, now-updated `adjList` — the algorithm automatically finds a new valid route around the closure, with no need to re-enter the graph or restart the program.

D) Algorithm Walkthrough

A - B (4)
A - C (1)
C - B (2)
B - D (5)
C - D (8)
D - E (3)

```
Enter number of roads: 6
Enter each road as: from to weight
A B 4
A C 1
C B 2
B D 5
C D 8
D E 3
```

```
Graph (Adjacency List):
A -> B(4) C(1)
B -> A(4) C(2) D(5)
C -> A(1) B(2) D(8)
D -> B(5) C(8) E(3)
E -> D(3)
```

Query: Find the shortest path from **A** to **E**.

Initialization: The distance of every node is set to ∞ (represented as `INT_MAX` in code), except the source node A, whose distance is set to 0. A min-heap priority queue is initialized with (0, A). A `parent[]` array (all values -1) is used to reconstruct the path once the algorithm finishes.

Step	Node Visited	dist[A]	dist[B]	dist[C]	dist[D]	dist[E]
Start	—	0	∞	∞	∞	∞
1	A	0	4	1	∞	∞
2	C	0	3	1	9	∞
3	B	0	3	1	8	∞
4	D	0	3	1	8	11
5	E	0	3	1	8	11

- **Step 1:** A is popped (distance 0). Its neighbors B and C are relaxed: $\text{dist}[B] = 4$, $\text{dist}[C] = 1$.
- **Step 2:** C has the smallest tentative distance (1), so it is popped next. Its neighbors are relaxed: $\text{dist}[B]$ improves from 4 to $1+2=3$, and $\text{dist}[D]$ becomes $1+8=9$.
- **Step 3:** B is popped (distance 3). Its neighbor D improves from 9 to $3+5=8$.
- **Step 4:** D is popped (distance 8). Its neighbor E is relaxed: $\text{dist}[E] = 8+3=11$.
- **Step 5:** E is popped (distance 11). It has no unvisited neighbors, and the queue empties.

Final Result: The shortest path is reconstructed by walking backward through `parent[]` from E: $E \rightarrow D \rightarrow B \rightarrow C \rightarrow A$, then reversed to give:

A -> C -> B -> D -> E

Total cost: 11

This matches the actual program output verified below.

F) Test Cases

Test Case 1 — Normal Case: Finding the Shortest Route

Objective: Verify that the program correctly finds the shortest path between two connected locations.

```
--- Menu ---
1. Find shortest path
2. Block a road (road closed)
3. Exit
Choice: 1
Enter source: A
Enter destination: E
Shortest path: A -> C -> B -> D -> E
Total cost: 11
```

Test Case 2 — Edge Case: Disconnected Graph

Objective: Verify that the program correctly handles a disconnected graph where no path exists between source and destination.

```
--- Menu ---
1. Find shortest path
2. Block a road (road closed)
3. Exit
Choice: 2
Enter road to block (from to): C B
Road between C and B is now blocked.
```

Test Case 3 — Complex Case: Dynamic Road Blocking

Objective: Verify that the program correctly updates the graph and recalculates the shortest path after a road is blocked, without rebuilding the graph.

Input:

- Graph: same 5-node graph as Test Case 1
- Step 1: Find shortest path A → E
- Step 2: Block road between C and B
- Step 3: Find shortest path A → E again

➔ **Actual Result:** blockRoad() removed the C–B edge from both adjacency lists, and dijkstra() correctly found the new optimal route without any graph rebuild. Test passed.

```
--- Menu ---
1. Find shortest path
2. Block a road (road closed)
3. Exit
Choice: 1
Enter source: A
Enter destination: E
Shortest path: A -> C -> D -> E
Total cost: 12
```

Test Case 4 — Invalid Node Input

Objective: Verify that the program validates user input and detects invalid location names.

Input:

- Source: XYZ (does not exist in the graph)
- Destination: E

Actual Result: The program used `find()` to check whether the entered name exists among the stored location names before running Dijkstra's Algorithm, and correctly rejected the invalid input. Test passed.

```
--- Menu ---
1. Find shortest path
2. Block a road (road closed)
3. Exit
Choice: 1
Enter source: XYZ
Enter destination: E
Invalid location name.
```

Test Case 5 — Same Source and Destination

Objective: Verify that the program correctly handles the case when source and destination are identical.

Input:

- Source: A, Destination: A

Actual Result: Since `dist[source]` is initialized to 0 before the algorithm runs, and the path-reconstruction loop stops immediately when `current == source` (i.e., `parent[A] == -1`), the program correctly returns the trivial path

```
--- Menu ---
1. Find shortest path
2. Block a road (road closed)
3. Exit
Choice: 1
Enter source: A
Enter destination: A
Shortest path: A
Total cost: 0
```

Evaluation Criteria (Rubrics)

Criterion	Marks
Correct implementation of Dijkstra's Algorithm	40
Viva	20
Lab report quality (design justification, trace, test cases)	25
Extension challenge (at least one attempted)	15
Total	100